USER MANUAL · ENGLISH EDITION

Terrain / O

Preliminary orienteering map editor: interface, tools, classification, vectorization and export, function by function and key by key.

Version 0.5.0 · 27 October 2026 · generated from docs/TUTORIAL.en.md

PRELIMINARY DRAFT — NOT VALID FOR COMPETITION

Contents

1. Before you start
2. Installation and commands
3. First run
4. Desktop, tablet and phone
5. The desktop shell
6. Panels
7. Import and georeferencing
8. Calibration, scale and print
9. Navigation: zoom, pan, rulers
10. Selection
11. Drawing
12. Editing features
13. Verification and provenance
14. Layers and map features
15. Classification
16. Vectorization
17. AI recognition and symbol layer
18. OpenStreetMap
19. DEM contour lines
20. Course and controls
21. History, undo and redo
22. Info and Navigator
23. Export
24. Language and theme
25. Keyboard reference
26. Limits and known issues

Terrain / O — Complete manual (English edition)

Version 0.5.0 · Preliminary draft

This manual documents every menu, tool, panel, shortcut and export. Read it end to end or keep it open as a reference while working.

> **Warning.** Terrain / O produces preliminary orienteering map drafts. It never declares runnability, vegetation under canopy, terrain safety, survey-grade elevation, invisible obstacles, a verified geographic north or IOF compliance. Verification is always an explicit user declaration, never an automatic result.

---

1. Before you start

Terrain / O is a working tool for preparing a draft map. It is not a certified race map and it does not replace fieldwork. Four provenance levels are always distinct:

- **Source image** — the original orthophoto, read-only, never modified.
- **Automatic** — classification, AI recognition, OSM and DEM output: experimental.
- **Manual** — geometry drawn or corrected by hand: unverified by default.
- **Verified** — only after you explicitly declare a field check. Any geometry edit revokes the declaration automatically.

On screen: automatic features get a thin dashed border, manual features a thin solid border, verified features a solid border plus a small ✓. The word *experimental* appears quietly in the Classification/Vectorization/AI panels, in Properties and once in the status bar. There is no question-mark marker and no permanent warning banner over the map.

2. Installation and commands

```
npm install
npm run dev           # http://localhost:5173
npm run build         # typecheck + production build
npm run preview
npm run typecheck
npm run lint
npm test
npx playwright test
npm run check         # all four quality gates
```

`npx playwright install chromium` is needed once for end-to-end tests.

3. First run

1. The app opens in the **essential workspace**: a full-window map with **Load**, **Run everything** and **Export**. **Advanced mode** opens the complete editor; **Essential** returns to the three-action screen. The choice is remembered.
2. Choose **Load** and select the image together with its world file and, when available, the `.prj`.
3. The source appears in the map. In advanced mode the left panel reports pixel or georeferenced measurements; if no world file is present the app says so and stays in pixels.
4. Open **Advanced mode** and use **Layers**, the tool palette and the Classification and Vectorization panels when you need manual control.
5. Export from **Export** in the essential bar, or from **File → Export map and data…** in advanced mode.

The first time advanced mode opens, the four-step onboarding tour appears. Skip it or follow it; **Help → Guided introduction** replays it.

4. Desktop, tablet and phone

The shell detects the viewport:

- **Desktop (> 900 px)** — full menu bar, options bar, vertical tool rail, both side docks and the status bar.
- **Tablet/phone (≤ 900 px)** — menu sheet (hamburger), floating panel buttons, drawers for the side panels and a bottom tool rail. Tap outside a drawer or use its close button to dismiss it.
- **Phone (≤ 620 px)** — compact status bar, larger touch targets and hidden secondary labels.

Touch gestures: one finger draws or pans according to the active tool, two fingers pan, pinch zooms. Double tap on the map is intentionally not bound, to avoid conflicts with system zoom.

5. The desktop shell

Menu bar — File, Edit, View, Tools, Classification, Vectorization, OSM, Export, Help. Every entry is functional; commands that this build cannot run are disabled with a tooltip. The right side shows the project name, the IT/EN language switch, import, export and help.

Options bar — parameters of the active tool: snap, grid and grid step, selection tolerance, polygon lasso, line offset, ruler units, zoom controls.

Tool palette — Selection, Drawing, Navigation, Measure and sample. Tooltips show the shortcut.

Map — raster source with a vector overlay, optional rulers, grid, crosshair, minimap, legend and a clean empty state before import.

Status bar — cursor pixel/WGS84 coordinates, zoom, scale, active tool, selection count and its single-word provenance, autosave state and storage estimate.

Dockable panels — drag a panel tab onto another dock to move it, drag the edge to resize, click the header button to collapse, double-click a tab to collapse the dock. The layout persists in localStorage under `terrain-o/ui-layout/v1`; **View** → **Reset panel layout** restores the defaults.

6. Panels

Panel	Contents
Project	source image, georeferencing, scale, snapping, provenance, local projects, storage, app info, rename
Layers	layer order, visibility, opacity, drag reorder
Features	searchable feature list with type and provenance
Properties	selected feature fields, geometry summary, origin line, verification
Classification	experimental line, parameters, live preview, stats, presets
Vectorization	experimental line, parameters, live overlay, counters, topology validation
AI recognition	local model, training sources, tree detection, symbol fusion report
History	labelled undo/redo stack and jump-to-state
Info	cursor coordinates in pixels/WGS84/projection, colour samples
Navigator	minimap with viewport rectangle
OSM	Overpass download, semantic fusion, automatic features, classification prior
DEM	contour download and parameters
Course	start, controls, finish, sequence, optimisation, GPX

7. Import and georeferencing

Select JPG/JPEG/PNG together with:

- world file: `.jgw`, `.jpgw`, `.pgw`, `.pngw`, `.wld`;
- projection: `.prj` or `.jpg.aux` (WKT).

Without a world file, measurements remain in pixels and OSM/DEM are disabled. Images above 4096 px are downscaled after confirmation; the original remains the source reference. Importing a new image asks for confirmation before replacing the current source. A `.terrain-o.json` project can also be imported; it is validated with Zod before adoption.

The source is re-encoded to JPEG q.85 only to limit local storage. Geometry is always expressed in the pixels of the imported image.

8. Calibration, scale and print

The working scale comes from georeferencing when available. Manual calibration has priority: choose **Calibrate 2 points**, click two points on the orthophoto and enter the real distance in metres. Additional measurements refine the scale by least squares.

Without calibration or georeferencing every measurement stays in pixels, and the interface, exports and metadata say so. The north arrow shows the source orientation only, never a verified geographic north. Choose the print scale (1:5 000 to 1:25 000) to see the paper size in millimetres.

9. Navigation: zoom, pan, rulers

- Wheel/pinch zooms around the cursor; `Ctrl+0` fits, `Ctrl+1` is 100 %, `Ctrl+=` / `Ctrl+-` step.
- Pan with the Hand tool (`H`), the middle button, or hold `Space` while dragging.
- Zoom tool (`Z`): click to zoom in, `Alt+click` to zoom out.
- Rulers: View → Rulers, unit px/m/degrees, with cursor indicators.
- Grid: View → Grid, step from the options bar or Project panel; snapping can use vertices, the grid, or both.
- Crosshair: View → Crosshair.
- Minimap: View → Minimap; click it to centre the view.

10. Selection

- **Object selection (`V`)** — click selects; drag moves; `Shift+click` adds or removes; a rectangle drag starting on empty map selects everything inside.
- **Rectangular selection (`M`)** — drag a marquee.
- **Lasso (`L`)** — freehand; enable **Polygon** in the options bar to click vertices and press `Enter`.
- The tolerance slider (`[` and `]`) controls how close a click must be to a geometry.
- `Ctrl+A` selects all features, `Ctrl+D` clears, arrows nudge by 1 px (`Shift` 10 px), `F` centres the selection.

11. Drawing

- **Area (`A`)** — click vertices; `Enter`, double-click or clicking the first vertex closes the polygon. `Esc` cancels the draft.
- **Line (`N`)** — click vertices, `Enter` finishes. The options bar offers a parallel line offset and **Apply** to offset the selected lines.
- **Point (`P`)** — click to place a point feature; on a course it becomes a control.
- **Text (`T`)** — reserved; disabled in this build.
- Snap indicators appear while drawing when snapping is on.
- Every geometry write goes through `updateFeatureGeometry`, which also revokes verification.

12. Editing features

Select one or more features and use the Properties panel to rename, change type, add notes or a course code. Geometry can be moved, nudged, extended with new vertices or deleted; `Ctrl+Shift+D` duplicates the selection (duplicates are never verified). Delete asks for confirmation and enters undo history.

13. Verification and provenance

In **Properties**, an automatically classified feature shows one grey line: *Source: automatic*. Manual features show *Source: manual*; verified features show *Source: verified*.

Declare verified opens a dialog with an optional note and a mandatory declaration checkbox. After confirmation the feature gets the ✓ marker and the status bar says *verified*. Editing its geometry revokes the declaration automatically, keeps the previous note in history with date and reason, and the status returns to *manual*.

14. Layers and map features

Layers, top to bottom by default: course and controls, map features, OSM (raw), contour lines, classification, source image. Each layer has visibility, opacity and order controls. Automatic features use a dashed border, manual ones a solid border, verified ones a solid border with ✓. The legend is optional and off by default.

15. Classification

The panel starts with one quiet line: *Experimental result*. Parameters:

- illumination tile size, gamma, L normalisation;
- SLIC compactness and superpixel size;
- colour/texture/edge weights;
- per-class L*a*b* thresholds;
- majority filter size and minimum area per class.

Changing a slider recomputes after 300 ms in the worker and paints a translucent live preview over the map. **Apply as automatic layers** commits the result, **Cancel** discards it, **Save preset** stores parameters in localStorage and can export them as JSON. Progress reports the active stage, tile and elapsed time; **Cancel** aborts cooperatively.

Classes are ISOM-aware and hierarchical: vegetation (forest dense/open, shrub, meadow, cultivated, bare ground), ground (rock, boulder, gravel, sand, mud), water (pond, stream, wetland, marsh), man-made (building, road, path, wall, fence) and an explicit *unclassified* state that is never hidden.

The classifier is a readable rule tree with per-pixel confidence; it is not a black-box model. A legend tab shows the class colours. The output is experimental material and never claims runnability or ground truth.

16. Vectorization

The pipeline is fully separate from classification and runs in a worker: mask reading (confidence 0.6 by default) → marching-squares contours → topology with holes and nested islands → Chaikin smoothing and Douglas–Peucker simplification with topology checks → adjacent-edge reconciliation → spurious geometry filter → export bridge.

Parameters: confidence threshold, Chaikin iterations, DP tolerance, minimum polygon area, minimum hole area, hierarchical priority and edge reconciliation. The panel shows polygon/hole/discarded counters, a live overlay toggle, presets and **Validate topology**. Validation checks closed rings, self-intersections, degenerate geometry and gaps/overlaps between adjacent polygons; any problem is listed and blocks export until fixed.

Every exported GeoJSON feature carries `class`, `classLabel`, `confidence`, `provenance: "auto"` and area when georeferenced. It never carries `provenance: "verified"`.

17. AI recognition and symbol layer

This path is separate from the rule classifier and always produces **automatic**, never verified, features. Open it from **AI** → **Open AI recognition panel** or the **AI** tab in the right dock.

How it learns

The model is a deterministic local random forest running in a web worker. It works on superpixels with 12 descriptors (L*a*b*, chroma, LBP texture, edge density/strength, segment area, fill, aspect, neighbour contrast and local variance) and predicts six classes: unclassified, vegetation, open, water, artificial and building.

Example source	Weight	Note
Manual area	3	strongest supervision: the user drew it
OSM area	0.5–1.1	external, may be outdated or misaligned
Rule pseudo-label	0.22–0.45	only where a real example is missing; can be disabled

Without OSM and manual areas the model can still start from pseudo-labels, but it only learns what the rules already know. For better results, draw a few representative building, forest, water and open areas, or import OSM before running recognition.

Controls

Control	Effect
Rule pseudo-labels	uses the most confident rule decisions as weak examples
Detect isolated trees	enables crown detection inside vegetation
Tree sensitivity	lowers or raises the detector threshold (10–100%)
Max trees	caps generated points (20–4000)
Boundary cleanup	majority filter 0–4 (0 = off)
Minimum area	discards components below this area in px^2

Recognize symbols reports progress by stage: illumination, features, superpixels, samples, training, classification, cleanup, vectorization. **Cancel** terminates the worker and applies nothing. At the end the panel shows samples, real examples, classes, holdout validation, areas, buildings, trees and feature importance.

Output

- Areas vectorized through the existing topology pipeline, including the building class.

- **Isolated tree** points detected on crowns.
- Every element has `source: 'ml'`, `mlConfidence` and a dashed automatic border.
- Nothing is verified, and no flow writes coordinates outside `updateFeatureGeometry`.

Holdout validation is agreement on examples excluded from training, not real-world accuracy. A high value never authorizes declaring anything verified.

Unifying the symbol layer

Fusion runs **automatically** after **Recognize symbols** and whenever OSM or vector features are generated; **Unify symbol layer** re-runs it on demand. It compares symbols of the same type:

- areas match at IoU ≥ 0.4 or containment ≥ 0.8;
- points — trees included — within 6 px are the same symbol;
- lines need at least 70% of both tracks within 3.5 px of each other, with comparable lengths;
- precedence is verified → manual/GPX → OSM → AI → vectorization.

Removed automatic ids are recorded in `mergedFrom` on the survivor. Manual and verified features are never removed or modified: if two protected features overlap, both are kept and the **Protected conflicts** counter reports it. Fusion never changes coordinates and does not touch the verification-revocation path.

One-click complete processing

AI → **Run the complete automatic pipeline**, or **Run everything** in the AI panel, runs in sequence:

1. OSM download, when the image is georeferenced and the option is enabled;
2. OSM feature generation (buildings, trees, water, green, roads, paths);
3. classification with the OSM prior;
4. vectorization through the topology pipeline;
5. AI recognition with pseudo-labels, buildings and trees;
6. DEM contour lines, when georeferenced and enabled;
7. symbol-layer fusion with area, line and point deduplication.

Every stage uses the previous results. If OSM or DEM is unreachable, or georeferencing is missing, that stage is skipped or reported as a warning and the pipeline continues with what is available. **Cancel** interrupts the current stage and keeps completed work. The final report lists OSM, classification, vector areas, AI symbols, buildings, trees, contours, merged duplicates and total symbols. The result is always automatic: it never declares trafficability, safety, survey-grade elevation or IOF conformity.

Limits and common problems

- It is not a pre-trained neural network: it only recognizes the supported classes and tree points, not every ISOM symbol.

- "More examples needed": draw areas of different classes, import OSM or enable pseudo-labels.
- "The model sees one class": add examples of other classes; one class cannot be separated.
- An outdated OSM building outranks AI during unification: correct or remove the OSM element for that case.
- A tree point is a geometric candidate, not a certainty: verify it in the field.

18. OpenStreetMap

Requires georeferencing. **Download OSM data** queries Overpass within the image bounding box for buildings, land use, water, parks, roads, paths, barriers, railways and trees. Multipolygons are reassembled and inner holes kept. Raw data appears as a separate, experimental layer. **Generate automatic features from OSM** creates unverified features for buildings, trees, water, forest, scrub, open ground, artificial surfaces, roads, paths, streams and barriers, deduplicated by OSM id. OSM trees become *Isolated tree* points and also feed the vegetation prior. The raw OSM layer is hidden after generation, leaving one deduplicated symbol layer; show it again from Layers to compare against the source. **Automatic fusion** generates the features and reclassifies in one action. Classification uses OSM boundaries as an optional per-superpixel prior; imagery keeps precedence when it is decisive. After classification the OSM panel reports overall agreement, image coverage and agreement per family. Areas below 65% are listed individually and clicking one selects the generated OSM feature. A low value flags outdated or misaligned OSM data without changing anything automatically. The list filters by **class** (vegetation, open, water, artificial) and by **minimum area** (100, 500 or 2,000 px^2); **Reset** restores all areas and **Export CSV** writes only the visible rows — name, OSM id, class, agreement, area and generation state — as UTF-8 with a BOM for Excel. Existing OSM features are not overwritten. Attribution: © OpenStreetMap contributors (ODbL).

19. DEM contour lines

Requires georeferencing. Contours come from public AWS Terrain Tiles (Mapzen Terrarium, about 30 m). Choose the interval and generate; each contour is stored with its elevation and displayed as an experimental layer. Elevation is indicative, not survey grade. Attribution travels with every export.

20. Course and controls

Add start, controls and finish from the Course panel. Controls are numbered in sequence; edit a control code inline. **Optimise controls** reorders them with nearest-neighbour + 2-opt without changing coordinates or verification state. The length is the sum of segments: it never declares runnability, climb or race time. Import GPX waypoints (requires georeferencing); export GPX/KML from the panel or the export dialog.

21. History, undo and redo

Every geometry and field change enters a labelled history. `Ctrl+Z` and `Ctrl+Shift+Z` / `Ctrl+Y` navigate it; the History panel shows labels and lets you jump to a state. Verification note and origin are preserved across undo/redo.

22. Info and Navigator

Info reports cursor position in pixels, WGS84 and EPSG:3857 when available, plus the current zoom, colour under the cursor and the classification colour. The Navigator shows a thumbnail with the visible rectangle; click to move.

23. Export

Format	Notes
PNG 1×/2×/3×	whole map, metric bar when scaled, attributions
SVG	real vector geometry, embedded raster, physical scale
PDF A4/A3	map, legend, warnings, attributions
OMAP XML v9	OpenOrienteering Mapper 0.9.x, complete official ISOM or ISSprOM set
GeoJSON	WGS84 if georeferenced, pixel coordinates + explicit metadata otherwise
GPX / KML	course and features, requires georeferencing
`.terrain-o.json`	complete project, including image and classification

Every file carries the preliminary-draft warning. OMAP output must be reviewed in OpenOrienteering Mapper by a cartographer. Topology validation runs before export; a failing geometry blocks the export with a clear message.

Map standard. In **Project** → **Scale and print**, choose **ISOM 2017-2** for forest maps or **ISSprOM 2019-2** for sprint maps. The selection changes canvas, legend, inspector and `.omap` symbol codes and geometry: road **503** becomes **501.17**, footpath **505** becomes **505.1**, stream **304** becomes **305**. The `.omap` file carries the complete official set (191 ISOM or 185 ISSprOM symbols) and its colour table, so Mapper opens the full palette. Water, paved areas and sprint buildings are written as fill plus companion outline objects, preserving holes. Definitions come from OpenOrienteering Mapper's GPL-3.0 symbol sets; see `NOTICE.md`.

24. Language and theme

The **IT/EN** switch in the menu bar changes the interface language and the document language, and remembers the choice. The complete Italian interface is the reference; English copy covers the shell, first-run flow and core panels, with the remaining specialist panels being translated progressively. The manuals exist in both languages (this file and `TUTORIAL.md`).

The interface is a compact graphite instrument with a single warm signal accent (reserved for the chrome, never used as a map symbol). Dark is the default theme for field work; **View** → **Light theme** switches to a paper desk (light grey canvas, off-white panels, ink text) for printing, daylight or screen sharing, so the map stays the deepest surface and the chrome never outshines it; **View** → **Dark theme** returns to the field theme. The choice is stored in `localStorage` (`terrain-o/ui-theme/v1`) and applied before the first paint, so the dark shell never flashes white on load. ISOM map colours are identical in both themes: only the frame changes, never the cartographic meaning. Both themes are checked for WCAG AA contrast by the Playwright/axe suite. Interface font: IBM Plex Sans/Mono (self-hosted); icons: Lucide (line, stroke 1.5px); coordinates use tabular figures. Focus rings are visible, tooltips include shortcuts and reduced-motion preferences are honored.

25. Keyboard reference

Key	Action
`V`	Object selection
`M`	Rectangular selection
`L`	Lasso
`A`	Area
`N`	Line
`P`	Point / control
`H`	Pan
`Z`	Zoom
`I`	Eyedropper
`T`	Text (not available)
`C`	Calibrate
`F`	Centre selection
`Esc`	Cancel
`Enter`	Close polygon / finish line
`Del`	Delete with confirmation
Arrows	Nudge 1 px (`Shift` 10 px)
`Space` + drag	Temporary pan
`Ctrl+Z`	Undo
`Ctrl+Shift+Z`, `Ctrl+Y`	Redo
`Ctrl+A` / `Ctrl+D`	Select all / deselect
`Ctrl+Shift+D`	Duplicate
`Ctrl+S`	Save now
`Ctrl+O` / `Ctrl+E`	Import / export
`Ctrl+0` / `Ctrl+1`	Fit / 100 %
`Ctrl+=` / `Ctrl+−`	Zoom step
`[` / `]`	Tolerance or line offset

Key	Action
`F1`	Help

26. Limits and known issues

No GeoTIFF import, no GPS navigation, no real-time collaboration. The OMAP symbol set is complete, but the editor only generates the classes supported by classification, AI, OSM and manual drawing; a cartographer must review the result. OSM and DEM data can be incomplete or misaligned. Automatic results are experimental. A verified state is a user declaration, not a certification by the software. Deleting browser data deletes local projects: keep `.terrain-o.json` exports.